THM | Campus Gießen | Wiesenstraße 14 | 35390 Gießen

Prof. Dr. Uwe Meyer
University of Applied Sciences

Praktische Informatik 2

Vorname		Nachname	
Matrikelnummer		Unterschrift	

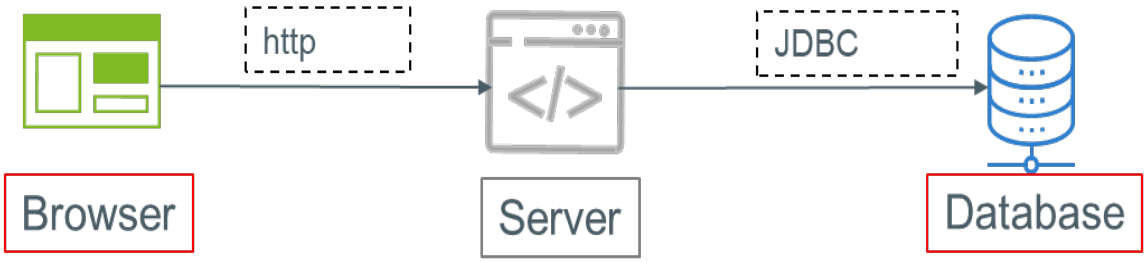
Nutzen Sie den vorgesehenen Platz unterhalb der Aufgaben oder schreiben Sie auf der Rückseite (bitte auf der Vorderseite kenntlich machen).
Schreiben Sie nicht mit Bleistift oder roter oder grüner Farbe.
Lassen Sie die Heftung der Blätter bitte intakt.
Legen Sie Ihren Studierendenausweis neben sich auf den Tisch.
Am Ende der Klausur bleiben bitte Sie auf Ihrem Platz sitzen – Ihre Klausur wird eingesammelt.
Als Hilfsmittel ist nur ein selbstgeschriebener DIN A4-Zettel zugelassen.
Sprechen mit dem Nachbarn oder Nutzung von Unterlagen oder Handy wird als Täuschungsversuch gewertet. Schalten Sie Ihr Handy aus und stecken Sie es in die Tasche.
Dauer: 90 Minuten.

Viel Erfolg!

	Maximale Punktzahl	Erreichte Punktzahl
Aufgabe 1	21	
Aufgabe 2	31	
Aufgabe 3	28	
Aufgabe 4	20	
Summe:	100	

Aufgabe 1 (4 + 5 + 7 + 5 Punkte):

a) Das folgende Architekturdiagramm stellt eine typische Architektur für Web-Anwendungen dar, wie Sie sie in den Übungen programmiert haben.
Geben Sie in den zwei Kästchen mit roten, durchgehenden Linien jeweils die Namen der Komponenten an.
Geben Sie in den zwei Kästchen mit schwarzen, gestrichelten Linien das verwendete Protokoll oder API an.



b) Eines der Ziele verteilter Systeme ist Offenheit. Erklären Sie dieses Ziel allgemein und nennen Sie mindestens 3 Stellen in der Architektur von Aufgabe a) an denen Offenheit unterstützt wird.

Unter Offenheit versteht man Interoperabilität (Fähigkeit mit Komponenten anderer Hersteller zusammenzuarbeiten) und Erweiterbarkeit.

Die Architektur unterstützt das Ziel Offenheit, da sie auf offenen und standardisierten Schnittstellen basiert.
Die Kommunikation zwischen Browser und Server erfolgt über HTTP, wodurch Clients unterschiedlicher Plattformen und Hersteller interoperabel sind.
Durch die Nutzung von JDBC ist der Server von einem konkreten Datenbanksystem entkoppelt, sodass Datenbanken ausgetauscht werden können.
Die Trennung von Präsentation, Anwendungslogik und Datenhaltung ermöglicht eine einfache Erweiterung und Austauschbarkeit einzelner Komponenten.

c) Ein Server stellt eine REST-Schnittstelle zur Verwaltung von Kursen bereit.
Jeder Kurs besitzt eine eindeutige ID, einen Namen und eine Teilnehmerliste.
Tragen Sie für jede der folgenden Tätigkeiten folgende Daten in die Tabelle ein:
- die Route (URI),
- das passende HTTP-Verb
- den HTTP-Statuscode im Erfolgsfall.

Tätigkeit	Route	Verb	Statuscode im Erfolgsfall
Alle Kurse anzeigen	/kurse	GET	200
Einen bestimmten Kurs anzeigen	/kurse/{id}	GET	200
Neuen Kurs anlegen	/kurse	POST	201
Namen eines bestehenden Kurses ändern	/kurse/{id}	PATCH	200 oder 204
Einen bestimmten Kurs löschen	/kurse/{id}	DELETE	204
Teilnehmer eines Kurses anzeigen	/kurse/{id}/teilnehmer	GET	200

d) Die Abkürzung REST steht für „Representational State Transfer“. Erläutern Sie, was hier mit „State“ (dt.: Zustand) gemeint ist.

Mit State ist hier der Zustand der Ressource gemeint.

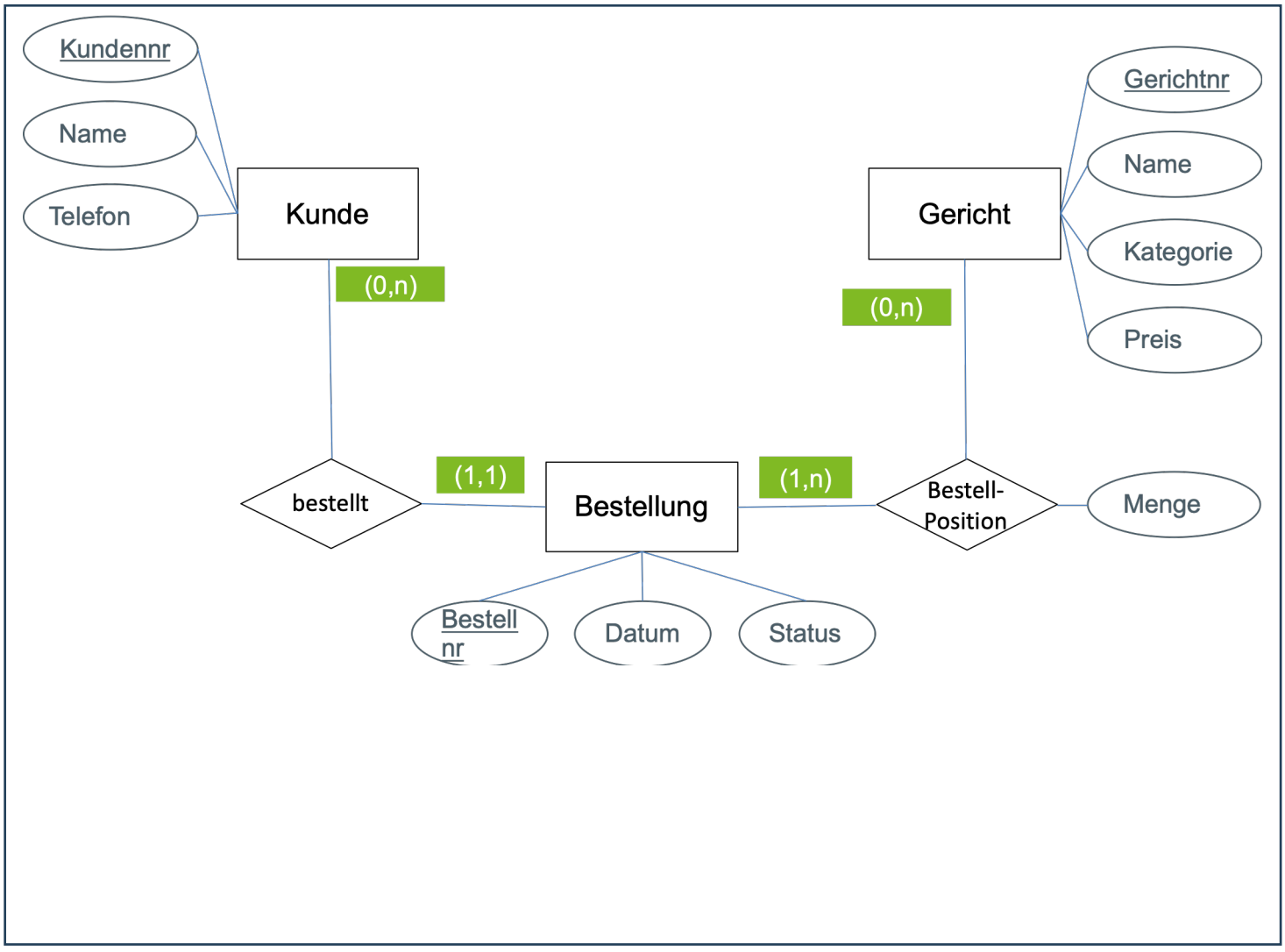
Bei einem GET wird der aktuelle auf dem Server gespeicherte Zustand zum Client übertragen, bei PUT/POST wird der Zustand der Ressource auf dem Server geändert.

Aufgabe 2 (8 + 4 + 4 + 3 + 5 + 4 + 3 Punkte)

a) Ein Lieferdienst für Essen möchte seine Bestellungen und Kunden verwalten. Zu diesen liegen folgende Daten vor:

- Kunden
  - Jeder Kunde hat eine Kundennummer, Name, Telefonnummer.
  - Ein Kunde kann mehrere Bestellungen aufgeben.
- Gerichte
  - Jedes Gericht hat eine Nummer, Name, Kategorie (z. B. Vorspeise, Hauptgericht, Dessert) und Preis.
  - Ein Gericht kann in mehreren Bestellungen enthalten sein.
- Bestellungen
  - Jede Bestellung hat eine Bestellnummer, ein Datum und einen Lieferstatus.
  - Eine Bestellung gehört genau einem Kunden.
  - Eine Bestellung kann mehrere Gerichte enthalten.
  - Für jedes Gericht in einer Bestellung soll die Menge gespeichert werden (z.B. dreimal Gericht27).

Zeichnen Sie ein entsprechendes ER-Diagramm und geben Sie an jeder Kante in (min, max)-Notation die Kardinalität an. Unterstreichen Sie die Primärschlüssel.



b) Ein Programmierer möchte Informationen zur Prüfungsanmeldung an Hochschulen speichern. Studierende sind genau in einem Studiengang eingeschrieben und können sich in jedem Semester für Prüfungen in Veranstaltungen anmelden. Ein Beispiel für die Tabelle „ANMELDUNG“ ist:

Matrikel-nummer	Name	Studiengang	Veranstaltungsnr	Veranstaltungstitel	Semester
1001	Müller	Informatik	INF101	Datenbanken	WS23
1001	Müller	Informatik	INF102	Programmieren	WS23
1002	Schmidt	Informatik	INF101	Datenbanken	WS23
1003	Meier	Wirtschafts-informatik	WIN201	BWL	SS24

Ist diese Tabelle in 1NF oder 2NF? Begründen Sie!

Die Tabelle ist in erster Normalform, da sie nur atomare Werte enthält.
Sie ist nicht in 2NF:
Ein sinnvoller Primärschlüssel ist (Matrikelnummer, Veranstaltungsnr, Semester).
Es existieren Abhängigkeiten:

- Name und Studiengang hängen nur von der Matrikelnummer ab
- Veranstaltungstitel hängt nur von der Veranstaltungsnr ab

Da es also nicht-Schlüsselattribute gibt, die nicht vom gesamten Primärschlüssel abhängen, ist die Tabelle nicht in 2NF (und damit auch nicht in 3NF).

c) Beschreiben Sie, welche Entitäten und Beziehungen Sie benötigen, um die o.a. Tabelle in 3NF zu überführen.

Die Tabelle muss in (mindestens) zwei Entitäten aufgeteilt werden:

- Studierende (Primärschlüssel Matrikelnummer, Name und Studiengang)
- Veranstaltung (Primärschlüssel Veranstaltungsnummer, Veranstaltungstitel und Semester).

Zusätzlich benötigt man eine Relation „Angemeldet“, die Studierende und Veranstaltung verknüpft.

d) Zur Speicherung der Gebäude und Räume gibt es zwei neue Tabellen „Gebaeude“ und „Raum“. Gebaeude hat die Spalten „ID“ (Primärschlüssel) und „Name“; Raum hat die Spalten „Raumname“ (Primärschlüssel), „Gebaeude_id“ (Fremdschlüssel zu „Gebaeude“) und „Kapazitaet“.
Erstellen Sie eine SQL Abfrage, die für alle Räume den Namen des Raums, den Namen des Gebäudes und Kapazität des Raums ausgibt.

```
select r.raumname, g.name, r.kapazitaet
from raum r, gebaeude g
where r.gebaeude_id=g.id;

oder:

select
   r.Raumname,
   g.Name,
   r.Kapazitaet
from Raum r
join Gebaeude g    on r.gebaeude_id = g.id;
```

e) Geben Sie eine Abfrage mit LEFT JOIN an, um für jedes Gebäude die Gesamtkapazitaet aller Räume im Gebäude ausgibt.

```
select g.name, sum(r.kapazitaet)
from gebaeude g left join raum r
on g.id=r.gebaeude_id
group by g.id;
```

f) Bei welcher Datenkonstellation würden bei dieser Abfrage ein LEFT JOIN und ein RIGHT JOIN unterschiedliche Ausgaben erzeugen?

Wenn es ein Gebäude ohne Räume gäbe, würde dieses Gebäude bei einem LEFT JOIN mit Kapazität NULL aufgeführt, bei einem RIGHT Join aber nicht.
Die umgekehrte Konstellation wäre nur möglich, wenn es Räume gäbe, die keinem Gebäude zugeordnet sind.

g) Erklären Sie den Unterschied zwischen einem FULL OUTER JOIN und einem INNER JOIN.

Ein INNER JOIN gibt nur die Datensätze zurück, für die es in beiden beteiligten Tabellen eine passende Übereinstimmung anhand der Join-Bedingung gibt. Datensätze ohne passenden Partner werden nicht angezeigt.
Ein FULL OUTER JOIN gibt alle Datensätze aus beiden Tabellen zurück. Existiert für einen Datensatz in der jeweils anderen Tabelle keine Übereinstimmung, werden die fehlenden Werte mit NULL dargestellt.

Aufgabe 3 (3 + 3 + 6 + 6 + 10 Punkte)

a) Folgende Ein- und Ausgabe wurden unter Linux/Unix/MacOS aufgezeichnet.

```
uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l
total 4
drwxr-xr-x  3 uwe uwe 4096 Sep 11 12:30 ./
drwxr-xr-x 14 uwe uwe 4096 Jul 16 17:30 ../
drwxr-xr-x  2 uwe uwe 4096 Sep 11 12:29 TESTS/
-rwxrw-r--  1 uwe uwe    0 Jun 11 11:09 a.txt*
lrwxrwxrwx  1 uwe uwe    5 Sep 11 12:30 tests -> TESTS/
```

Geben Sie die Permissions von `TESTS`, `a.txt` und `tests` in numerischer/oktaler Form an:

TESTS	754
a.txt	764
tests	777

b) Ein Prozess P sei jeweils im Zustand ‚running'. Bitte geben Sie für die folgenden Ereignisse an, in welchen neuen Zustand der Prozess wechselt.

Ereignis	Neuer Zustand
Die P zustehende Zeitscheibe ist abgelaufen, sodass ihm die CPU entzogen wird.	Ready
P wartet auf eine Benutzereingabe.	Waiting
Prozess beendet sich selbst.	Exit

c) Was versteht man in Betriebssystemen unter „interner Fragmentierung“ und warum ist es ein Problem?

Interne Fragmentierung tritt auf, wenn das Betriebssystem den Speicher nur in festen Größen (Pages) vergibt, aber ein Prozess weniger Speicher benötigt und dadurch der restliche Speicher in einer Page nicht verwendet werden kann. Dadurch entstehen im Speicher Lücken und es wird mehr Speicher verbraucht als eigentlich nötig.

d) Erklären Sie, welche Nachteile entstehen würden, wenn ein Unix-/Linux-Dateisystem keine inodes oder eine andere zentrale Datenstruktur zur Verwaltung von Metadaten der Dateien verwendet.

Ohne eine entsprechende Datenstruktur für die Metadaten müssten diese direkt in den Dateien gespeichert (oder berechnet) werden, was den Zugriff verlangsamt.
Um Informationen über ein Verzeichnis zu erlangen, müsste man jede Datei innerhalb des Verzeichnisses lesen, was sehr langsam sein wird.
Ohne die Verweise auf die Cluster, die den Inhalt der Datei speichern, wäre die Speicherverwaltung ineffizient und langsam.
Daten könnten verloren gehen, wenn die Metadaten in den Dateien inkonsistent sind.
Informationen werden evtl. mehrfach redundant gespeichert.

e) Gegeben sind vier Prozesse mit folgenden Eigenschaften:

Prozess	Ankunftszeit	Rechenzeit
P1	0s	7s
P2	1s	4s
P3	1s	6s
P4	1s	5s

Das Betriebssystem verwendet Round-Robin-Scheduling mit Zeitscheiben der Länge 2s**.** Zur Vereinfachung nehmen wir an, dass der Prozesswechsel 0s dauert. Tragen Sie für jeden Prozess die Endzeiten, die Verweildauer (ab dem Ankunftszeitpunkt) und die Wartezeiten ein.

Prozess	Endzeit	Verweildauer	Wartezeit
P1	22	22	15
P2	12	11	7
P3	20	19	13
P4	21	20	15

Aufgabe 4 (5 + 4 +11 Punkte)

a) Was versteht man unter einem „Rainbow-Table-Angriff“ und welche Maßnahme kann diesen verhindern/erschweren?“

Rainbow Tables sind vorgefertigte Tabellen von Passwort-Hashes, sodass ein Angreifer diese nur mit der Passwort-Tabelle vergleichen müsste.

Ein Salt ist ein zufälliger Wert, der vor dem Hashen an das Passwort angehängt oder kombiniert wird. Durch die Zufälligkeit des Salts müsste für jedes mögliche Salt eine neue Tabelle erstellt werden. Das macht den Angriff praktisch unmöglich, da Speicher- und Rechenaufwand sehr stark steigen.

b) Welche beiden Verschlüsselungsverfahren werden bei SSL/TLS verwendet und warum?

SSL/TLS verwendet zunächst asymmetrische Verschlüsselung, um einen gemeinsamen Schlüssel auszuhandeln. Die asymmetrische Verschlüsselung ist hier notwendig, weil auf sichere Art ein Schlüssel ausgetauscht werden muss.

Mit diesem gemeinsamen Schlüssel werden danach die Daten symmetrisch verschlüsselt, da dies schneller ist als asymmetrische Verschlüsselung.

c) Ein Java-Programm verarbeitet eine Benutzereingabe, die aus einem HTML-Formular kommt. Die `eingabe` wird an die Methode `verarbeiteEingabe` weitergegeben, die wie folgt lautet:

```
public void verarbeiteEingabe(String eingabe) {
    char[] buffer = new char[8];

    for (int i = 0; i < eingabe.length(); i++) {
        buffer[i] = eingabe.charAt(i);
    }
}
```

(i) Erklären Sie, was man unter einem Buffer Overflow (Speicherüberlauf) versteht.

(ii) Begründen Sie, warum der obige Code ein Sicherheitsproblem darstellt, obwohl der Speicher durch Java verwaltet wird.

(iii) Nennen Sie zwei Maßnahmen, mit denen dieses Problem vermieden werden kann.

(i) Ein Buffer Overflow liegt vor, wenn ein Programm mehr Daten in einen Speicherbereich schreibt, als dafür vorgesehen sind, und dadurch benachbarte Speicherbereiche überschreibt.

(ii) Im gegebenen Code wird über die Länge der Eingabe iteriert, ohne zu prüfen, ob der Index die Größe des Buffers überschreitet.
Ist die Eingabe länger als das Array, kommt es zu einem Zugriff außerhalb des erlaubten Bereichs, was in Java eine Exception (z.B. `ArrayIndexOutOfBoundsException`) auslöst, der zu einem Programmabbruch führen könnte. Die Availability des Programms wird also verletzt.

(iii)
- Längenprüfung vor dem Schreiben in den Buffer
- Nutzung sicherer Methoden bzw. dynamischer Datenstrukturen (z. B. `String` oder List).